

# SMI3.0.5

## Technical Validation Report

---

### DRAFT - CODE-BASED REVIEW

This draft follows the structure and validation-report style of the SMI2.0v3.5 Technical Validation Report. It is grounded in the SMI3 source code and configuration files provided for review. Statements requiring independent execution, production evidence, business documentation, data-source certification, or validation results are clearly identified as pending and should not be treated as completed validation conclusions.

## 1 Executive Summary

### 1.1 Preamble

SMI3 represents a redesigned mortgage-insurance modelling and implementation framework using an AWS SageMaker pipeline, automated preprocessing, AutoGluon/XGBoost model training, model evaluation, explainability outputs, loss-ratio calculations, model registration, and real-time inference components. The submitted code identifies the inference version as SMI 3.0.5.

The technical validation review focuses on the implementation evidenced in the submitted CI/CD code package. The review considers data quality and suitability controls, feature engineering and feature selection, conceptual soundness, model training/evaluation design, implementation consistency, reproducibility controls, and deployment-related logic.

### 1.2 SMI3.0.5 Key Updates / Design

- Automated policy- and borrower-level preprocessing with Snowflake and static S3 data inputs.
- Application-level borrower aggregation and credit-score selection/binning logic.
- Construction of a binary target from funded mortgage performance outcomes, including delinquency and claim events.
- Use of macroeconomic features including HPI and unemployment transformations.
- Integration of an external LGD component into policy-level processing and loss-ratio calculations.
- AutoGluon-based binary classification restricted by configuration to XGBoost, with ROC AUC as the configured evaluation metric.
- Bagging/cross-validation configuration, out-of-fold predictions, feature importance, SHAP explainability, and risk-ranking outputs.
- Separate training, test, and recent populations produced by preprocessing.
- SageMaker pipeline orchestration for preprocessing, training, evaluation, conditional checks, and model registration.
- Real-time JSON validation, preprocessing, scoring, and loss output generation.

### 1.3 Technical Validation Scope

#### 1.3.1 In Scope

- Review of SMI3 preprocessing and application-level dataset construction.
- Review of data cleansing, missing-value handling, duplicate handling, filtering, joins, and feature-range controls.
- Review of target definition and training/test/recent population construction.
- Review of borrower aggregation, credit-score selection, feature transformations, and macroeconomic feature construction.

- Review of leakage controls and separation of analysis-only variables from model inputs.
- Review of model methodology, training configuration, bagging/cross-validation design, feature importance, and SHAP implementation.
- Review of model evaluation, risk-ranking, lift/gain, and validation artifact generation.
- Review of SageMaker pipeline orchestration, configuration, model registration, and inference implementation.
- Review of code-level reproducibility controls, logging, and MLflow artifact capture.

### 1.3.2 Out of Scope / Pending Evidence

- Independent verification of the accuracy and completeness of source data in Snowflake, S3, external credit-score files, economic files, or the external LGD model.
- Independent execution of the complete SMI3 pipeline and replication of developer model results.
- Independent statistical performance conclusions because executed output artifacts and final metric results were not provided with the code package.
- Independent assessment of privacy, legal, ethical, or customer-data permissions; these require governance evidence outside the submitted code.
- Business approval of target definitions, population exclusions, economic-scenario assumptions, and LGD methodology.
- Production-versus-development result reconciliation and production monitoring evidence.
- Final model risk rating, residual risk acceptance, and model approval decision.

## 1.4 Technical Validation Summary

**1. Data processing:** The code contains structured policy and borrower processors, explicit column validation, duplicate handling, controlled joins, missing-value treatments, feature clipping/range logic, and logging of population sizes. These are positive implementation controls; however, their empirical impact must be independently quantified.

**2. Population design:** Training is restricted to funded mortgages with BOOKYEAR 2009-2015, while testing uses EFFECT\_SID 202101-202312. A separate recent population is also created. The long temporal separation supports out-of-time assessment, but representativeness and the rationale for excluded periods require validation evidence.

**3. Target definition:** The target is derived from delinquency/claim categories. No-event observations are assigned target 0, while event categories are generally assigned target 1; 'Others' are assigned missing. The business definition and treatment of claim-only/delinquency-only cases should be formally confirmed.

**4. Leakage controls:** Shared utilities identify leaky identifiers and temporal/metadata fields for exclusion, and training code explicitly checks for remaining leaky columns. This is a strong design control that should be independently tested against the final feature matrix.

**5. Model methodology:** Configuration specifies a binary AutoGluon model restricted to XGBoost with ROC AUC as the evaluation metric, five bagging folds, fixed random seed, and no weighted ensemble. The conceptual rationale for XGBoost-only selection and the absence of HPO in the submitted configuration should be documented and challenged.

**6. Explainability:** Feature importance and SHAP artifacts are implemented. Validation should assess whether feature effects are stable, economically sensible, and appropriate for mortgage-insurance use rather than relying only on generated plots.

**7. Evaluation:** The code generates predictions, leaderboard metadata, risk-ranking, lift/gain and category-capture artifacts. Final conclusions on performance, calibration, stability, and acceptance thresholds require executed results.

**8. MLOps / deployment:** The pipeline supports preprocessing, training, evaluation and registration using configured Docker images and MLflow/SageMaker artifacts. Final deployment assurance requires independent pipeline execution and environment-to-environment replication.

### 1.5 Model Fit for Use

A final fit-for-use conclusion is not issued in this draft. Based on code review alone, the SMI3 implementation contains a number of appropriate technical controls for preprocessing, leakage prevention, training reproducibility, explainability, evaluation, and pipeline orchestration. Final fit-for-use should be conditional on successful independent result replication, validation of data suitability and representativeness, confirmation of target/population design, performance and calibration testing, review of the LGD dependency, and closure or acceptance of validation findings.

## 2 Technical Review of SMI3

### 2.1 Data Quality and Suitability

#### 2.1.1 Data Sources and Lineage

Policy and borrower data are loaded from Snowflake, while economic, claim/delinquency, geographic and credit-score inputs are read from static S3 files. The code provides identifiable processing paths from raw sources into policy and borrower processors and then merges outputs by APPLICATION\_NUMBER using a one-to-one validation rule. Validation should obtain a formal source-to-model lineage document and reconcile a sample of records back to authoritative source systems.

#### 2.1.2 Accuracy, Completeness and Data Controls

The policy processor validates required columns, records duplicate counts, removes duplicate application records, aggregates duplicate claim episodes to application level, and applies feature-range controls. The JSON inference validator separately enforces required fields, nullability, data types, allowed values and numeric bounds. Validation should independently profile missingness, duplicates, invalid values, range exceptions and reconciliation errors, and quantify records affected by each control.

#### 2.1.3 Missing Values and Data Cleansing

The code applies explicit missing-value treatments, including filling missing claim/delinquency indicators with zero, filling certain financial/property variables using fallback fields, and dropping rows missing selected model variables before population construction. Validation should assess whether each treatment is economically justified and whether dropped observations create material selection bias.

#### 2.1.4 Duplicate and Aggregation Controls

Policy data are deduplicated by APPLICATION\_NUMBER and claim episodes are aggregated to application level using maximum delinquency/claim indicators and summed incurred loss. Borrower data are aggregated to application level using maximum values and summary statistics such as minimum, maximum, average, median and weighted average. Validation should confirm that these aggregation choices preserve intended risk information and do not overstate or understate borrower-level risk.

#### 2.1.5 Training Population and Representativeness

The training population is ultimately restricted to funded mortgages, non-missing credit score, and BOOKYEAR 2009-2015. The test population uses EFFECT\_SID 202101-202312. Validation should compare development, test, recent and current production populations across major risk drivers and explain the treatment of the intervening years, including the explicit exclusion of 2020 claim episodes in policy processing.

### 2.1.6 Timeliness and Economic Data

The model constructs economic features including HPI and unemployment changes. For the test population, HPI\_+4 and UE\_+4 are mapped using a December 2022 regional reference; the recent population uses the maximum effective period as reference. Validation should confirm the business purpose of this design, ensure no future-information leakage is introduced, and assess whether the timing convention matches intended model use.

### 2.1.7 Synthetic and Proxy Data

The submitted code does not provide clear evidence of synthetic-data generation. Proxy-style transformations may exist, including property-value fallback logic, economic mappings and external LGD-derived fields. Validation should document all proxy variables, their rationale, source accuracy, stability and impact. Synthetic data should be documented as not applicable if none are used.

### 2.1.8 Data Compliance

Privacy, regulatory, ethical and internal data-use compliance cannot be established from the submitted source code. Validation should obtain evidence of approved data usage, access controls, third-party data permissions, retention requirements and fairness/proxy-variable review.

## 2.2 Feature Engineering and Feature Selection

### 2.2.1 Target Definition

The policy processor creates categories including Declined, Others, Delq\_Claim, Claim\_Only, Delq\_Only and No\_Event. The binary target assigns No\_Event to 0, Others to missing, and the remaining categories to 1. Validation should confirm that this definition is consistent with the approved PD/claim-risk objective, observation/performance horizon and intended use.

### 2.2.2 Borrower-Level Feature Engineering

Borrower-level processing selects among multiple credit-score sources using a defined priority and score bins, and aggregates borrower attributes to application level. Validation should reproduce score selection and aggregation for sample applications and assess whether maximum/average/weighted-average choices are conceptually appropriate.

### 2.2.3 Policy-Level Feature Engineering

The model constructs and/or processes LTV, debt-service ratios, amortization, property characteristics, product variables, geographic variables, economic-region mappings, HPI and unemployment features, and LGD-related fields. Validation should independently reproduce material transformations and confirm units, cut-offs, fallback rules and production consistency.

### 2.2.4 Feature Ranges and Outliers

The policy processor defines explicit ranges for selected variables such as property value, LTV, property age, loan term, TDSR, GDSR, income, employment and credit variables. Validation should confirm whether values are clipped, excluded or otherwise transformed, quantify affected observations, and assess sensitivity to the chosen bounds.

### 2.2.5 Leakage Prevention

The shared utility defines leaky identifiers and temporal/metadata columns, and training/evaluation code removes analysis-only and leaky columns from model input. Validation should independently inspect the final model feature list and verify that no target-derived, post-outcome, future, identifier or operational metadata fields remain.

## 2.2.6 Feature Stability and Selection

The submitted code computes feature importance and SHAP values, but the code package alone does not demonstrate a formal documented feature-selection study covering stability, multicollinearity/redundancy, alternative feature sets, or fairness/proxy concerns. These analyses should be obtained or independently performed before final validation conclusion.

## 2.3 Conceptual Soundness

### 2.3.1 Model Purpose and Methodology

The configuration defines a binary classification problem with target='target', ROC AUC as the evaluation metric, and XGBoost as the allowed model family within AutoGluon. Validation should confirm the exact business interpretation of the probability output and why XGBoost is appropriate relative to simpler and alternative challenger methodologies.

### 2.3.2 Training and Cross-Validation

The submitted configuration uses a fixed random seed of 42 and five bagging folds, one bagging set, no stacking, and no weighted ensemble. Training uses the training dataset directly and does not provide a separate tuning dataset in the submitted implementation. Out-of-fold predictions are generated for validation utilities. Validation should assess whether this design is sufficient for model selection and whether an independent holdout/out-of-time dataset remains untouched.

### 2.3.3 Hyperparameter Optimization

The submitted configuration sets the HPO trial count to zero, indicating that automated hyperparameter optimization is disabled in the reviewed configuration. Validation should obtain the rationale for the final XGBoost hyperparameters and evidence that reasonable alternatives were assessed.

### 2.3.4 Overfitting, Stability and Robustness

The temporal separation between training and test data provides a basis for out-of-time assessment. The code also produces a recent dataset. Validation should compare training/out-of-fold, test and recent performance, assess degradation, segment stability and economic-period sensitivity, and determine whether the model remains robust under changed portfolio conditions.

### 2.3.5 Explainability and Economic Reasonableness

Permutation feature importance and SHAP outputs are implemented. Validation should assess direction, magnitude and stability of major feature effects and challenge counterintuitive relationships, especially for credit score, LTV, debt-service ratios, HPI, unemployment and property characteristics.

### 2.3.6 LGD Integration and Loss Calculation

The policy processor references an external LGD model package and the loss-ratio calculator combines LGD and model probabilities to generate loss-related outputs. The external LGD methodology and its validation are not evidenced by the submitted code package and should be reviewed separately or cross-referenced to an approved LGD validation.

### 2.3.7 Reproducibility

The training code fixes Python/NumPy random seeds and logs configuration and artifacts to MLflow. Docker images are explicitly configured in the pipeline. However, the requirements file generally uses minimum package versions rather than exact pins. Validation should confirm that production builds are immutable and reproducible through image digests/tags, lock files or equivalent controls.

## 2.4 Model Diagnostics

### 2.4.1 Discrimination and Risk Ranking

The evaluation framework generates probability predictions and risk-ranking outputs, including lift, gain and category-capture artifacts. Executed ROC AUC and rank-ordering results should be independently replicated.

### 2.4.2 Calibration

The submitted code package does not by itself demonstrate a complete PD calibration assessment. Validation should compare predicted probabilities with observed event rates overall and by score/risk bands, time, geography and material segments.

### 2.4.3 Out-of-Time Testing

The 2021-2023 test population is materially later than the 2009-2015 training population. Validation should independently reproduce OOT metrics and investigate performance changes due to underwriting, economic, data and portfolio shifts.

### 2.4.4 Sensitivity and Stress Behaviour

Validation should test how predictions change when major inputs are varied and confirm economically sensible behaviour, particularly for LTV, credit score, debt-service ratios, HPI and unemployment.

### 2.4.5 Segment Performance

Performance should be assessed by province/region, product, lender/channel where available, LTV band, credit-score band, property type, vintage and other material portfolio segments.

## 2.5 Model Documentation

The code contains inline documentation, configuration comments, logging and artifact generation. A final validation conclusion requires a formal model-development document describing purpose, scope, target, data lineage, exclusions, transformations, assumptions, methodology, hyperparameters, limitations, performance, explainability, monitoring and deployment. The report should reconcile that documentation to the reviewed implementation.

## 3 Independent Code Review

### 3.1 Independent Code Replication Environment

The submitted implementation is designed for AWS SageMaker and uses configured custom Docker images, MLflow tracking, Snowflake inputs, S3 static data, and SageMaker model artifacts. Independent validation should execute the pipeline in a validation-controlled environment using the same immutable code and image versions and compare generated artifacts with developer outputs.

### 3.2 CI/CD and Locked-Down Code

The pipeline code supports parameterized infrastructure and model registration. The configuration references specific Docker image tags. Validation should obtain Git commit IDs, CI/CD build evidence, image digest/version evidence, access-control evidence, and confirmation that the code validated is the code promoted to production.

### 3.3 Code Review Observations

- Positive control: one-to-one merge validation is used when combining policy and borrower datasets.
- Positive control: duplicate application and claim-episode handling is explicitly logged and processed.
- Positive control: leaky and analysis-only columns are explicitly excluded from model inputs.
- Positive control: fixed random seeds and MLflow logging support reproducibility.
- Positive control: feature importance, SHAP, risk-ranking and validation artifacts are generated.

- Validation challenge: training population is limited to 2009-2015 funded business; representativeness versus current business requires evidence.
- Validation challenge: 2020 claim episodes are excluded in policy processing; rationale and impact should be quantified.
- Validation challenge: test economic variables are standardized to a December 2022 regional reference; rationale, leakage implications and intended-use consistency should be confirmed.
- Validation challenge: HPO is disabled in the submitted configuration; model-selection and hyperparameter rationale should be documented.
- Validation challenge: dependency versions are generally lower-bounded rather than fully pinned; immutable build controls should be evidenced.

## 4 Independent Result Replications

Independent result replication is pending. The following replications should be completed before final report issuance:

- Reproduce preprocessing row counts and key population distributions.
- Reproduce target/category counts and selected feature transformations.
- Reproduce final training feature matrix and verify leakage exclusions.
- Reproduce model training using the validated code/configuration and compare model metadata.
- Reproduce out-of-fold, test and recent predictions.
- Reproduce ROC AUC and other agreed performance metrics.
- Reproduce lift/gain/risk-ranking and SHAP/feature-importance artifacts.
- Reproduce loss-ratio calculations that combine model probability and LGD.
- Compare outputs across Development, QA/Validation and Production or pre-production environments.
- Confirm identical code, configuration, model artifact and dependency/image versions across environments.

## 5 Technical Model Governance

### 5.1 Validation Findings / Action Items

| ID         | Preliminary Validation Item                                           | Status | Required Evidence / Action                                                                                                     |
|------------|-----------------------------------------------------------------------|--------|--------------------------------------------------------------------------------------------------------------------------------|
| SMI3-TV-01 | Training population representativeness and exclusion-period rationale | Open   | Compare development/OOT/recent/current portfolio distributions; document 2009-2015 training choice and treatment of 2016-2020. |
| SMI3-TV-02 | Target definition and event-category treatment                        | Open   | Confirm approved business definition, horizon, treatment of Delq_Only, Claim_Only, Delq_Claim, Declined and Others.            |
| SMI3-TV-03 | Missing-value and row-exclusion impact                                | Open   | Quantify records removed/imputed and assess selection bias and model impact.                                                   |
| SMI3-TV-04 | Economic feature timing / December 2022 test reference                | Open   | Document rationale and demonstrate no inappropriate future-information leakage.                                                |
| SMI3-TV-05 | Feature selection and XGBoost/hyperparameter rationale                | Open   | Provide challenger analysis, stability/redundancy assessment and rationale for disabled HPO.                                   |
| SMI3-TV-06 | LGD dependency                                                        | Open   | Provide approved LGD                                                                                                           |

|            |                                                     |         |                                                                                            |
|------------|-----------------------------------------------------|---------|--------------------------------------------------------------------------------------------|
|            |                                                     |         | model/version, validation status, interface reconciliation and sensitivity.                |
| SMI3-TV-07 | Independent performance and calibration replication | Pending | Execute validation pipeline and reproduce OOF/OOT/recent metrics and calibration.          |
| SMI3-TV-08 | Environment/code/artifact replication               | Pending | Reconcile Git commit, image digest, configuration and model artifacts across environments. |
| SMI3-TV-09 | Dependency reproducibility                          | Open    | Evidence immutable production image/build or exact dependency lock mechanism.              |
| SMI3-TV-10 | Privacy/fairness/data-governance evidence           | Pending | Obtain governance approvals and assess sensitive/proxy variables.                          |

## 5.2 Model Deployment

The SageMaker pipeline implements a sequence covering preprocessing, training, evaluation and registration, with conditional quality/evaluation logic represented in the pipeline code. The inference service validates JSON inputs, preprocesses application data, loads the AutoGluon model, generates probability predictions, combines outputs with LGD/loss calculations and returns versioned results. Independent deployment validation should confirm parity between batch/development transformations and real-time inference transformations.

## 5.3 Model Monitoring

The submitted code generates evaluation and risk-ranking artifacts but does not by itself establish the complete production monitoring framework. Final validation should confirm monitoring thresholds and escalation for data quality, missingness, feature drift, population drift, prediction drift, discrimination, calibration, segment performance, LGD/loss performance, exceptions and model changes.

# 6 Appendix

## 6.1 Source Files Reviewed

- \_utils.py
- borrower\_processor.py
- evaluate.py
- get\_pipeline\_definition.py
- image\_utils.py
- jsonprocessor.py
- jsonvalidator.py
- lr\_calculator.py
- model\_result.py
- pipeline.py
- policy\_processor.py
- predictor.py
- preprocess.py
- requirements.txt
- run\_pipeline.py
- shap\_utility.py
- train.py
- train\_config.yaml
- utils.py

- SMI2.0v3.5 LPM - Technical Validation Report (format/reference baseline)

## 6.2 Key Code-Evidenced Configuration

| Item                     | Reviewed Configuration / Code Evidence        |
|--------------------------|-----------------------------------------------|
| Problem type             | Binary classification                         |
| Configured model family  | XGBoost via AutoGluon                         |
| Evaluation metric        | ROC AUC                                       |
| Random seed              | 42                                            |
| Bagging folds            | 5                                             |
| Bagging sets             | 1                                             |
| Stacking                 | 0 levels in submitted train_config            |
| Weighted ensemble        | Disabled                                      |
| HPO                      | 0 trials / disabled in submitted train_config |
| Training period          | BOOKYEAR 2009-2015; funded records            |
| Test period              | EFFECT_SID 202101-202312                      |
| Explainability           | Permutation feature importance and SHAP       |
| Tracking                 | MLflow                                        |
| Pipeline platform        | AWS SageMaker                                 |
| Inference version string | SMI 3.0.5                                     |

## 6.3 Finalization Requirements

This draft should be finalized only after independent execution evidence is available. Numeric performance results, final findings, severity ratings, management responses, closure status, and the final fit-for-use recommendation should be inserted from validation-owned evidence rather than inferred from source code.